IMDB Movies -Data Analysis using SQL

➢ Introduction

This project focuses on analyzing the IMDB movies dataset using SQL. The dataset consists of two main tables: movies and directors. The goal is to perform data exploration, cleaning, and analysis to extract meaningful insights related to movies, directors, popularity, revenue, and ratings.

➢ Database Overview

The database used is:

- Database Name: project_movie_database
- Tables Used:
  - movies
  - directors

➢ Attribute Information

- Table Directors:
  - Name: Name of the Director
  - ID: Unique ID of the Directors
  - Gender: Gender of the Director→ 0/2=Male, 1=Female
  - Department: Department of the Directors.
- Table Movies:
  - ID: Unique Id for Movies
  - Original title: Movie name
  - Budget: Budget of the movie
  - Popularity: Popularity of the Movie
  - Release Date: Release date for the movies
  - Revenue: Revenue collected movie
  - Title: Initial title of the movie
  - Vote Average: Average IMDB rating
  - Vote Count: Number of Vote the movie got
  - Overview: Description of the movie
  - Tagline: Tagline of the Movie
  - UID: Unique ID for Movie
  - Director ID: Director ID

➢ Problem Queries

1. **Data Understanding**

```
use project_movie_database;
show tables;
desc movies;
desc directors;
```

Explanation:

- View available tables
- Understand table structure
- Identify column names and data types

2. Retrieve Data

```
Select * from movies;
Select * from directors;
```

Explanation:

- Retrieves complete data from both tables
- Helps in initial exploration

3. Count Total records

```
Select count(*) as Total_movies from movies;
```

Explanation:

- Counts total number of movies
- Used as a basic KPI

4. Data Quality Check(Null Values)

```
selectcount(distinct
id,original_title,budget,popularity,release_date,revenue,title,vote_average,vote_count,overvie
w,tagline,uid,director_id)
as unique_count,
sum(CASE WHEN id IS NULL THEN 1 ELSE 0 END) AS null_id,
sum(CASE WHEN original_title IS NULL THEN 1 ELSE 0 END) AS null_original_title,
sum(CASE WHEN budget IS NULL THEN 1 ELSE 0 END) AS null_budget,
sum(CASE WHEN popularity IS NULL THEN 1 ELSE 0 END) AS null_popularity,
sum(CASE WHEN release_date IS NULL THEN 1 ELSE 0 END) AS null_release_date,
sum(CASE WHEN revenue IS NULL THEN 1 ELSE 0 END) AS null_revenue,
sum(CASE WHEN title IS NULL THEN 1 ELSE 0 END) AS null_title,
sum(CASE WHEN vote_average IS NULL THEN 1 ELSE 0 END) AS null_vote_average,
sum(CASE WHEN vote_count IS NULL THEN 1 ELSE 0 END) AS null_vote_count,
sum(CASE WHEN overview IS NULL THEN 1 ELSE 0 END) AS null_overview,
sum(CASE WHEN tagline IS NULL THEN 1 ELSE 0 END) AS null_tagline,
sum(CASE WHEN uid IS NULL THEN 1 ELSE 0 END) AS null_uid,
sum(CASE WHEN director_id IS NULL THEN 1 ELSE 0 END) AS null_director_id
from movies;
```

Explanation:

- Identifies missing values in each column
- Ensures data reliability

5. Filter Specific Directors

```
select * from directors
where name in('James Cameron','Luc Besson','John Woo');
```

Explanation:

- Filters specific directors using IN
- Useful for targeted analysis

6. Pattern Matching

```
select * from directors
where name like 's%';
```

Explanation:

- Filters Finds directors whose names start with ‘S’
- Uses LIKE operator

7. **Count Female Directors**

```
select count(name) as Female_directors from directors
where gender = 1;
```

Explanation:

- Counts number of female directors
- Gender = 1 represents female

8. **Ranking (10th Female Director)**

```
select name from directors
where gender =1 limit  1 offset 9;
```

Explanation:

- Retrieves the 10th female director
- Uses LIMIT and OFFSET

9. **Top 3 Popular Movies**

```
select original_title,popularity
from movies
order by popularity desc limit 3;
```

Explanation:

- Sorts movies by popularity
- Retrieves top 3

10. **Top 3 Bankable Movies**

```
select original_title,revenue
from movies
order by revenue desc limit 3;
```

Explanation:

- Identifies highest revenue-generating movies
- Retrieves top 3

11. **Highest Rated Movie After 2000**

```
select * from movies
where  date_format(release_date,'%m %d %y') >='01-01-2000'
order by vote_average desc limit 1;
```

Explanation:

- Filters movies after 2000
- Finds highest rated movie

12. **Which movie(s) were directed by Brenda Chapman**

```
SELECT m.original_title,d.name
FROM movies m JOIN directors d
ON m.director_id = d.id
WHERE d.name = 'Brenda Chapman';
```

Explanation:

- Combines movies and directors tables
- Retrieves movies directed by a specific director
- Uses JOIN for relational mapping

13. **Director with Most Movies**

```
SELECT d.name, COUNT(*) AS total_movies
FROM movies m JOIN directors d
ON m.director_id = d.id
GROUP BY d.name ORDER BY total_movies DESC LIMIT 1;
```

Explanation:

- Groups movies by director
- Finds director with highest number of movies

14. Most Bankable Director

```
select d.name,sum(m.revenue) as Highest_revenue
from movies m
join directors d
on m.director_id = d.id
group by d.name order by Highest_revenue desc limit 1;
```

Explanation:

- Calculates total revenue per director
- Identifies most profitable director

➢ Conclusion

This project demonstrates how SQL can be used for:

- Data exploration
- Data cleaning
- Aggregation and filtering
- Joining multiple tables
- Extracting business insights